



Módulo 6

Patrones Avanzados y Producción

Curso: Terraform hands-on para GCP

120 min

Dynamic blocks · for_each · data sources · migración zero-downtime · DR

Objetivos de Aprendizaje

- Aplicar `dynamic blocks` para generar bloques repetitivos (firewall, IAM, network interfaces).
- Usar `for_each` y `count` correctamente, entendiendo cuándo cada uno aplica.
- Consumir `data sources` para descubrir recursos existentes en lugar de hardcodear IDs.
- Refactorizar el state con `terraform state mv`, `terraform import` y bloques `moved {}` (1.1+).
- Planificar y ejecutar una migración de infraestructura con cero downtime siguiendo el patrón estándar (replicar, validar, cutover, apagar).
- Actualizar providers de forma segura respetando `lifecycle { create_before_destroy }` y ventanas de mantenimiento.

0. Apertura y objetivos

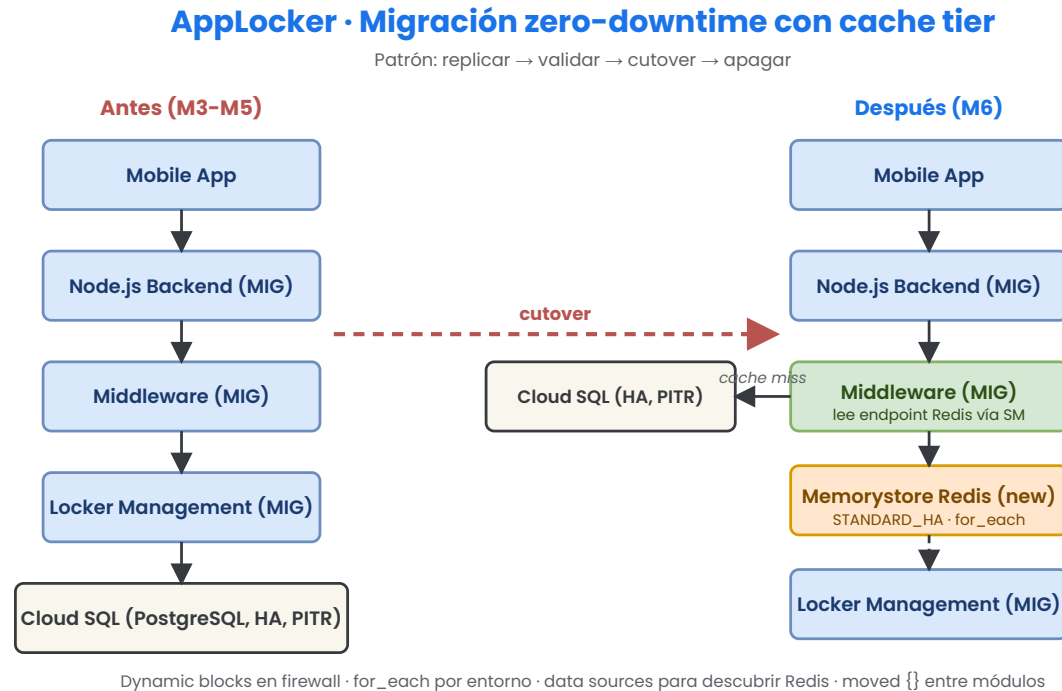
5 min · ¿qué pasa cuando la realidad cambia más rápido que el código?

Dónde estamos al llegar a M6

- **M1:** bucket de state remoto importado y versionado en GCS.
- **M2:** módulo `cloud-sql` publicado en el Private Module Registry.
- **M3:** VPC 3-tier, Managed Instance Groups, Cloud SQL privado con HA, firewall por tags.
- **M4:** service accounts dedicadas, secretos en Secret Manager, labels de coste, backups y PITR.
- **M5:** pipeline GitOps con Terraform Cloud + GitHub Actions + Workload Identity Federation + OPA.

Pregunta del día: la infraestructura ya es código y está bajo control de cambios. ¿Qué pasa cuando **la realidad cambia más rápido que el código?**

Caso AppLocker — punto de partida



Problema: el equipo de locker-mgmt reporta latencia alta en los picos de apertura de cerraduras.

Decisión de plataforma: intercalar un tier de cache (Memorystore Redis) entre Middleware y Locker Management.

Reto: hacerlo **sin que AppLocker deje de responder ni un segundo**, pasando por el pipeline GitOps del M5.

1. Dynamic blocks, loops y for_each

≈ 8 min · repetición parametrizada y lifecycle

¿Qué resuelven los `dynamic blocks`?

- Repetición de bloques anidados (`ingress` , `egress` , `network_interface` , `iam_binding` , `container` , `volume_mount`).
- La forma del bloque varía en función de variables (puertos, tags, descripciones).
- Sintaxis: `dynamic "nombre_bloque" { for_each = <colección> content { ... } }`.
- El iterador por defecto es `nombre_bloque.value` (configurable con `iterator`).

Errores comunes: iterar sobre un `set` esperando preservar orden (los sets no lo garantizan); usar `for_each` con claves que solo se conocen tras un `apply` .

Ejemplo: firewall parametrizado con `dynamic "allow"`

```
variable "firewall_rules" {
  type = map(object({
    description = string
    ports       = list(string)
    source_tags = list(string)
  }))
}

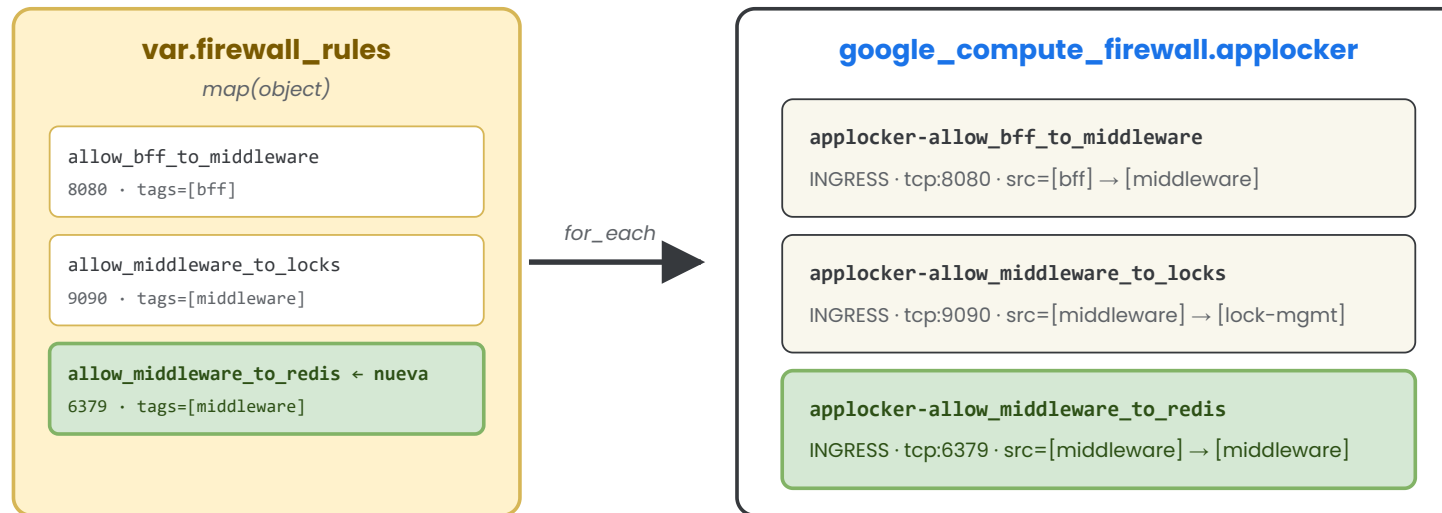
resource "google_compute_firewall" "applocker" {
  for_each = var.firewall_rules

  project      = var.project_id
  name         = "applocker-${each.key}"
  network      = var.network_self_link
  description  = each.value.description
  direction    = "INGRESS"

  source_tags = each.value.source_tags
  target_tags = ["middleware"]
}
```

De N reglas hardcodeadas a 1 parametrizada

Dynamic blocks: de N reglas hardcodeadas a 1 parametrizada



Una sola línea en el mapa = una regla nueva en el plan. **Sin tocar HCL del recurso.**

Beneficio: añadir la regla `allow_middleware_to_redis` es una línea nueva en el mapa. El plan propone el `+ create` correspondiente sin tocar HCL del recurso.

`count` **VS** `for_each`

Aspecto	<code>count</code>	<code>for_each</code>
Índice	entero (<code>count.index</code>)	clave (<code>each.key</code>)
Cambios de tamaño	shifta índices → recursos reemplazados	solo añade/quita → in-place
Identidad	por posición	por clave
Cuándo usarlo	réplicas idénticas de tamaño variable	conjunto nombrado (mapa o set)

Regla práctica del curso: si vas a poder **añadir o quitar elementos sin recrear el resto**, usa `for_each`. Si vas a poder **cambiar el tamaño de un grupo de réplicas idénticas**, usa `count`.

Ejemplo: Memorystore con `for_each` por entorno

```
resource "google_redis_instance" "applocker_cache" {
  for_each = var.environments

  project      = var.project_id
  name         = "applocker-cache-${each.key}"
  tier          = each.value.tier          # STANDARD_HA en prod, BASIC en dev
  memory_size_gb = each.value.memory_gb
  region       = each.value.region
  redis_version = "REDIS_7_2"

  authorized_network = each.value.network_self_link
  connect_mode       = "PRIVATE_SERVICE_ACCESS"

  labels = {
    app      = "applocker"
    env      = each.key
    managed  = "terraform"
  }
}
```

for expressions y lifecycle

- `[for x in list : expr]` y `{ for k, v in map : new_k => new_v }` para aplanar colecciones en outputs y locals.
- Caso real del lab: aplanar el mapa de instancias Redis a un mapa de `host:port` consumible por el middleware.
- `optional()` y `try()` (Terraform 1.3+) toleran outputs nulos durante el cutover.

```
lifecycle {  
  create_before_destroy = true  
}
```

create_before_destroy: Terraform crea el recurso nuevo **antes** de destruir el viejo. Imprescindible para migraciones sin downtime y para upgrades críticos de provider.

2. Data sources

≈ 7 min · descubrir recursos existentes sin recrearlos

¿Cuándo usar una `data source`?

- Para **leer** recursos existentes que no gestionas en este stack: recursos de otro módulo, otra cuenta, otro equipo, o creados manualmente.
- Si el recurso existe y no quieres recrearlo → **data source**.
- Si quieres que Terraform sea la fuente de verdad → **resource**.

Regla de oro del curso: si existe y no quieres recrearlo, `data "..."`. Si Terraform debe poseerlo, `resource "..."`.

Data sources clave del curso

```
data "google_project" "current" {
  project_id = var.project_id
}

data "google_redis_instance" "applocker_cache" {
  for_each = var.environments
  name     = "applocker-cache-${each.key}"
  region   = each.value.region
}

data "google_compute_network" "applocker" {
  project = var.project_id
  name    = "applocker-vpc"
}

locals {
  redis_hosts = {
    for k, r in data.google_redis_instance.applocker_cache : k => r.host
  }
}
```

Otras: `google_compute_subnetwork`, `google_secret_manager_secret_version`, `google_sql_database_instance`.

Dependencias implícitas vs `depends_on`

- Terraform construye el grafo de dependencias **automáticamente** a partir de las referencias entre recursos y data sources.
- **Anti-patrón:** añadir `depends_on = [...]` cuando ya hay una referencia. Hace el código más frágil y opaco.
- **Cuándo SÍ usar `depends_on`:** cuando la dependencia es por un atributo que **no se referencia** (p. ej. un side-effect sobre IAM que no devuelve valor).

Una `data source` **refresca en cada plan**. Si alguien cambia un atributo fuera de Terraform, el siguiente `plan` lo detecta → red de seguridad contra drift.

3. Estrategias de migración de state

≈ 8 min · mover, importar y refactorizar sin recrear

```
terraform state mv
```

- Mueve un recurso de una dirección a otra **dentro del state** sin tocar la infraestructura.
- Útil para mover entre módulos (`module.cloud_sql.x` → nuevo módulo) o renombrar sin destruir.
- Sintaxis: `terraform state mv <origen> <destino> .`
- Antes de tocar nada: `terraform plan` y `terraform refresh` para confirmar que el state está sano.

Riesgo: no queda rastro en git. El siguiente dev abre el PR y no entiende por qué el state cambió.

terraform import

- Trae un recurso existente al state bajo una dirección nueva.
- Limitaciones: hay que escribir **a mano** el bloque `resource`; no todos los recursos son importables; algunos campos requieren `refresh` posterior.
- **Flujo recomendado:**
 1. Escribir el bloque `resource {}` con los atributos conocidos.
 2. `terraform import <address> <id_gcp>`
 3. `terraform plan` para ver qué difiere.
 4. Ajustar el código hasta que el plan sea **no-op**.

Bloques `moved {}` (Terraform 1.1+)

```
moved {  
  from = google_redis_instance.applocker_cache["prod"]  
  to   = module.cache.google_redis_instance.applocker_cache["prod"]  
}
```

- Declara en código que un recurso cambió de dirección.
- Terraform **re-escribe el state automáticamente** al planificar.
- **Ventaja sobre** `state mv`: queda en el repositorio, en el PR, versionado.
- En el lab: al mover Redis del módulo raíz a `module.cache`, dejamos el `moved {}` en el módulo raíz para que el primer plan no intente recrearlo.

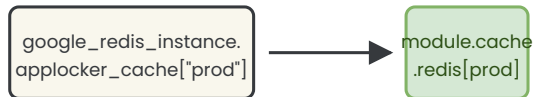
Regla: preferir `moved {}` sobre `state mv` cuando sea posible.

terraform state mv vs bloques moved {}

Refactor de state: state mv vs bloques moved {}

terraform state mv

operación sobre el state, fuera de git



```
$ terraform state mv \  
google_redis_instance.x module.cache.x
```

No queda rastro en git

Riesgo: el siguiente dev no sabe
por qué el state cambió

Bloques moved {} (1.1+)

declarado en el .tf, versionado en git

```
moved {  
  from = google_redis_instance  
        .applocker_cache["prod"]  
  to   = module.cache  
        .google_redis_instance  
        .applocker_cache["prod"]  
}
```

Queda en el PR · revisable

Terraform re-escribe el state
automáticamente al planificar

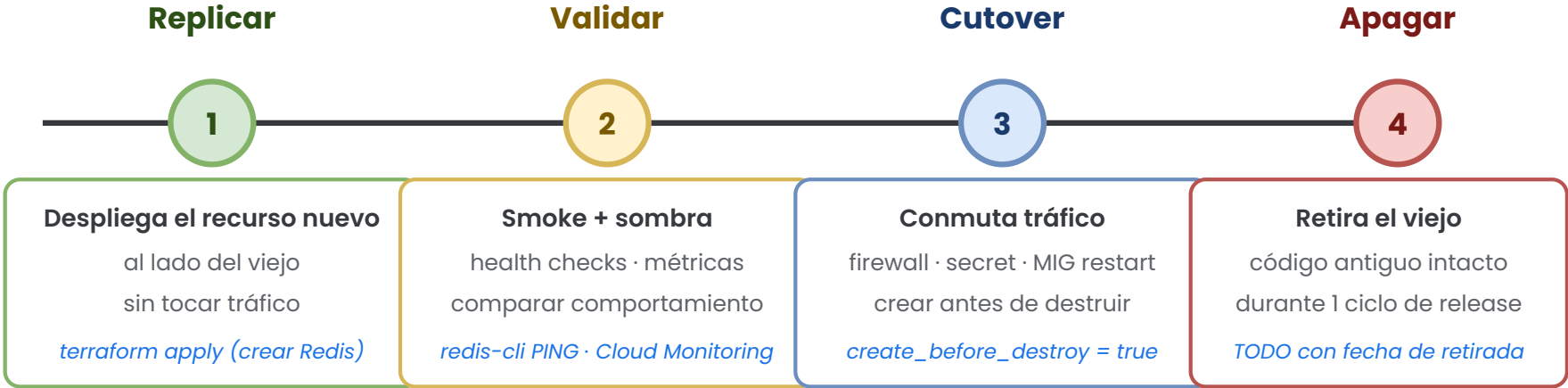
Regla del curso: **preferir moved {}** sobre state mv cuando sea posible

`terraform state rm` y el patrón zero-downtime

- `state rm` : saca un recurso del state **sin destruirlo** en la nube. Útil para traspasar propiedad a otro stack. *Pedir doble confirmación y dejar nota en el runbook.*
- **Patrón zero-downtime — 4 pasos, en este orden, siempre:**
 - 1. Replicar:** desplegar el recurso nuevo **al lado** del viejo.
 - 2. Validar:** health checks, smoke tests, sombra.
 - 3. Cutover:** conmutar tráfico (firewall, endpoint, DNS).
 - 4. Apagar:** eliminar el viejo una vez validado el nuevo en producción.

Patrón zero-downtime visual

Patrón zero-downtime en 4 pasos



Regla del curso: nunca saltarse un paso · siempre validar antes de cutover

Regla: nunca saltarse un paso. Validar **siempre** antes de cutover.

4. Upgrades de provider y disaster recovery

≈ 7 min · pinning, ventanas y runbooks

Tipos de bump de provider

Tipo	Ejemplo	Riesgo	Acción
Patch	1.5.0 → 1.5.3	Bajo	Bugfixes. Generalmente seguro.
Minor	1.5 → 1.6	Medio	Nuevas features. Revisar changelog.
Major	5.x → 6.x	Alto	Breaking changes. Ventana de mantenimiento + refactor.

- Pin en `required_providers { google = { version = "~> 6.0" } }`.
- `terraform init -upgrade` solo cuando se decide bumpear conscientemente.
- Probar primero en `dev` (recordar el `for_each` por entorno del módulo nuevo).
- Para evitar downtime: `lifecycle { create_before_destroy = true }` + ventana + plan revisado en PR + `terraform plan -out=tfplan` + `terraform apply tfplan`.

Disaster Recovery por componente

- **Cloud SQL** (caso AppLocker): PITR ya activado en M4. Validar que el bucket de backups está en **otra región**.
- **Memorystore**: tier `STANDARD_HA` replica a una zona secundaria; failover automático. Si la pérdida afecta a toda la región, restaurar desde backup o recrear.
- **State remoto (GCS)**: el versioning del bucket (activado en M1) es nuestra red de seguridad. Si el state se corrompe, restaurar una versión anterior.
- **Runbooks**: en el repositorio, en `docs/runbooks/`, con la cadena exacta de comandos.

Plantilla de runbook de incidente

INCIDENTE: <título>

FECHA: <YYYY-MM-DD HH:MM UTC>

IMPACTO: <servicios afectados>

PASO 1 – Diagnóstico

```
$ gcloud sql operations list --instance=applocker-sql-prod --limit=5
$ gcloud redis instances describe applocker-cache-prod --region=us-central1
```

PASO 2 – Mitigación inmediata

(definir aquí: failover, rollback del último apply, etc.)

PASO 3 – Restauración

```
$ terraform plan -out=dr.tfplan
$ terraform apply dr.tfplan
```

PASO 4 – Verificación

```
$ curl -fsS https://<endpoint>/health
```

PASO 5 – Post-mortem

```
terraform force-unlock
```

- Libera un state lock cuando un `apply` quedó colgado (equipo perdido, conexión cortada, crash).
- **Peligro:** si el lock está realmente en uso, dos applies concurrentes **corrompen el state**.
- **Regla del curso:**
 1. Verificar que **ningún otro proceso** corre contra ese state (`terraform plan -lock-timeout=1s`).
 2. Confirmar con otro miembro del equipo.
 3. `terraform force-unlock <LOCK_ID>` (el ID aparece en el error).

5. Lab integrador: migración zero-downtime

≈ 80 min · añadir cache tier a AppLocker sin tirar el servicio

Prerrequisitos

- Recursos de M3-M5 desplegados y verificados.
- Acceso al repositorio del curso con permisos de PR.
- `gcloud`, `terraform >= 1.5`, `gh` CLI autenticados.
- Variable `TF_VAR_project_id` apuntando al proyecto GCP del curso.
- Pipeline GitOps del M5 sigue verde (último `apply` OK).

Paso 1 · Crear Memorystore con `for_each`

1. Crear el módulo `modules/cache` con `variables.tf`, `main.tf` y `outputs.tf`.
2. Declarar el módulo en la raíz pasando la VPC y los entornos:

```
module "cache" {
  source = "../modules/cache"

  project_id      = var.project_id
  network_self_link = module.network.vpc_self_link

  environments = {
    dev = { tier = "BASIC",      memory_gb = 1, region = "us-central1" }
    prod = { tier = "STANDARD_HA", memory_gb = 5, region = "us-central1" }
  }
}
```

Commit en rama `feature/m6-cache-tier`. PR al pipeline. El `terraform plan` debe proponer **crear** dos instancias, sin destruir nada.

Paso 2 · Consumir el endpoint con `data sources`

```
data "google_redis_instance" "applocker_cache" {
  for_each = module.cache.instance_addresses

  name    = "applocker-cache-${each.key}"
  region = each.value.region
}

locals {
  redis_endpoint = {
    for k, r in data.google_redis_instance.applocker_cache : k => "${r.host}:${r.port}"
  }
}
```

- Añadir al módulo de red un output `vpc_self_link` (ya debería existir de M3; verificar).
- Exponer `redis_endpoint` como variable de entorno del middleware vía Secret Manager (patrón M4).

Paso 3 · Refactorizar firewall con `dynamic blocks`

```
resource "google_compute_firewall" "applocker" {
  for_each = var.firewall_rules

  project      = var.project_id
  name         = "applocker-${each.key}"
  network      = var.network_self_link
  description  = each.value.description
  direction    = "INGRESS"

  source_tags = each.value.source_tags
  target_tags = each.value.target_tags

  dynamic "allow" {
    for_each = each.value.ports
    content {
      protocol = "tcp"
      ports    = [allow.value]
    }
  }
}
```

Añadir `allow_middleware_to_redis` en el mapa. Verificar en el plan: la regla nueva aparece como `+ create`, las existentes como `~ update in-place`.

Paso 4 · Aplicar con zero-downtime

Replicar → **Validar** → **Cutover** → **Apagar**

- 1. Replicar:** merge del PR `feature/m6-cache-tier` → el plan crea Redis en `dev` y `prod`.
- 2. Validar:** smoke que conecte a `applocker-cache-prod:6379` desde una VM bastion y haga `PING`. Comprobar métricas en Cloud Monitoring.
- 3. Cutover:** nuevo PR `feature/m6-cutover-middleware` que añade la regla de firewall, actualiza el Secret Manager con el endpoint y reinicia el MIG del middleware.
- 4. Apagar:** dejar el código antiguo intacto al menos un ciclo de release. Marcar `TODO` con fecha de retirada.

Paso 5 · Simular DR (siempre en `dev`)

⚠ El lab se ejecuta siempre contra el entorno `dev`, nunca contra `prod`.

```
# Snapshot manual de dev
gcloud redis instances export applocker-cache-dev \
  --gcs-location=gs://${TF_VAR_state_bucket}-snapshots/ \
  --async

# Borrar dev (simula pérdida)
gcloud redis instances delete applocker-cache-dev --region=us-central1

# Terraform recrea la instancia (gracias al state)
terraform plan
terraform apply
```

Cargar el snapshot exportado con `gcloud redis instances import ...`.

Paso 6 · Validación final y limpieza

- `terraform plan` en `main` debe ser **no-op**.
- Verificar en TFC que la última ejecución es verde.
- Comprobar `labels` correctos (`app=applocker` , `env=prod|dev` , `managed=terraform`).
- Confirmar que **no se ha destruido** ningún recurso de M3-M5 (`terraform state list` + consola GCP).
- Ejecutar el smoke test end-to-end desde la app móvil simulada.

Limpieza: dejar `prod` tal cual. Vaciar y borrar el bucket de snapshots del paso 5 (o documentar para siguiente edición).

6. Cierre del curso, Q&A y siguientes pasos

5 min · recap completo de los 6 módulos

Recap completo del curso

- **M1 — State y Remote Backend:** bucket remoto, locking con GCS, import, workspaces.
- **M2 — Arquitectura Modular:** módulo Cloud SQL publicado y versionado.
- **M3 — IaC Core:** VPC 3-tier, MIG, Cloud SQL privado con HA.
- **M4 — Seguridad:** SAs dedicadas, Secret Manager, labels, backups, PITR.
- **M5 — CI/CD:** pipeline GitOps con TFC + GHA + WIF + OPA.
- **M6 — Patrones Avanzados:** dynamic blocks, data sources, migración zero-downtime, DR.

Recursos para seguir aprendiendo

- [Documentación oficial de Terraform](#)
- [Google Cloud Provider](#)
- [Terraform Best Practices](#)
- [HashiCorp Learn — Terraform en GCP](#)
- [Google Cloud Architecture Framework](#)
- *Terraform Up & Running* (3ª ed., Yevgeniy Brikman) — lectura recomendada.

Certificaciones recomendadas

- **HashiCorp Certified: Terraform Associate** (003) — siguiente paso natural tras este curso.
- **Google Cloud Professional Cloud Architect** — perfil multi-cloud / arquitectura.
- **Google Cloud Professional Cloud DevOps Engineer** — profundizar en SRE + GitOps en GCP.

Resumen del Módulo 6

Puntos Clave

- **Dynamic blocks + for_each**: una sola parametrización genera N recursos (firewall, IAM) y tolera crecimiento sin recreaciones.
- `count` **VS** `for_each`: por posición vs por clave. Si vas a añadir/quitar, usa `for_each`.
- **Data sources**: descubrir recursos existentes sin hardcodear IDs. Refrescan en cada plan → red anti-drift.
- **Refactor de state**: preferir bloques `moved {}` (1.1+) sobre `state mv`; `state mv` para emergencias; `import` para traer recursos al state.
- **Zero-downtime**: replicar → validar → cutover → apagar. `lifecycle { create_before_destroy = true }` en recursos críticos.
- **Upgrades de provider**: pin de versión, probar en `dev`, ventana de mantenimiento para major bumps.
- **DR**: PITR + bucket cross-region + snapshots de Memorystore + versioning del state + runbooks en el repo.

Referencias

Recursos Adicionales

- [Dynamic blocks — Terraform docs](#)
- `count` — [meta-argument](#) · `for_each` — [meta-argument](#)
- [Lifecycle](#) `create_before_destroy`
- [Data sources](#)
- `terraform state mv` · `terraform import`
- [Bloques](#) `moved {}`
- [Provider versions](#)
- [Cloud SQL DR \(PostgreSQL\)](#)
- [Memorystore for Redis](#)
- [Terraform Cloud](#)
- [GitHub Actions + Workload Identity Federation](#)



¡Curso completado!

Módulo 6 finalizado

Gracias por la energía durante los 3 días · Encuesta de satisfacción al salir